COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	7/24/26	**DATE SUBMITTED:**	7/24/26
NAME:	Reyes, Cyrill Joferson B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

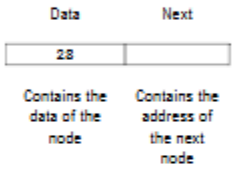
1. PROCEDURE OUTPUT

5. Procedure

ILO A: Construct C++ code for a singly and doubly linked list in C++

A.1. Singly Linked List

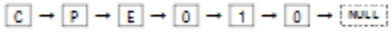
To start, we will do a simple implementation of a linked list. We must keep in mind the visual representation of the individual nodes that will make up our linked list:



Every node in a singly linked list will have 2 compartments, the data and the pointer to the next. The data contains the element of a single type that we want it to contain. The link will point to the next node in our list. In C++, the node is defined as:

```
class Node{
public:
    char data;
    Node *next;
};
```

We will implement a list to represent the string "CPE010" that looks like the figure below:



Implementation will follow the given steps:

1. Create the node pointers and initialize as NULL
2. Create new instances of the node class and allocate them in the heap
3. Define every node in the list
4. Point the last node to null

Simple implementation:

```
#include<iostream>
#include<utility>

class Node{
public:
    char data;
    Node *next;
};

int main(){
    //step 1
    Node *head = NULL;
```

```
        Node *second = NULL;
        Node *third = NULL;
        Node *fourth = NULL;
        Node *fifth = NULL;
        Node *last = NULL;

        //step 2
        head = new Node;
        second = new Node;
        third = new Node;
        fourth = new Node;
        fifth = new Node;
        last = new Node;

        //step 3
        head->data = 'C';
        head->next = second;

        second->data = 'P';
        second->next = third;

        third->data = 'E';
        third->next = fourth;

        fourth->data = 'O';
        fourth->next = fifth;

        fifth->data = 'L';
        fifth->next = last;

        //step 4
        last->data = 'O';
        last->next = nullptr;
    }
```

Although we have created the linked list, this is an implementation that is useless and meant only to show what we want to happen for the given output. **Run your code and screenshot the output, then briefly discuss how the output came to be and how could it be improved (table 3-1)?**

Imagine if we had to make multiple items in the list in the hundreds or thousands! This would be too tedious, ineffective, and inefficient. So, we must implement certain methods. These methods/operations associated with the linked lists are:

- Traversal
- Insertion at head
- Insertion at any part of the list
- Insertion at the end
- Deletion of a node

Linked List Traversal

```
Algorithm: ListTraversal (parameter: pointer to node n)
WHILE n IS NOT EQUAL TO null
      PRINT data OF n
      GO TO NEXT NODE n := next
      ENDWHILE
```

```
    PRINT next line
    END
```

Sometimes we have a linked list, and we need to insert a node somewhere other than at the end of the list. We will look at a couple of different ways to insert a node into an existing list.

To insert a node at the head:

1. Allocate memory for the new node
2. Put our data into the new node
3. Set Next of the new node to point to the previous Head
4. Reset Head to point to the new node

To insert a node at any location between the head and tail:

1. Check if it is the head node (previous node is null)
2. If null, print "Previous node cannot be null."
3. Allocate a new node
4. Store data in the new node
5. Point new node to the node previous node was pointing to
6. Point previous node to the new node

To insert a node at the end:

1. Allocate new node
2. Dereference to the head node
3. Store data in new node
4. Point next of new node to NULL
5. Traverse the list until next of the node is null
6. Point the next of the current node to the new node

To delete a node from linked list:

1. Find previous node of the node to be deleted.
2. Change the next of previous node.
3. Free memory for the node to be deleted.

Create code for each of the pseudocode given for all list operations above.
Provide screenshots in table 3-2.

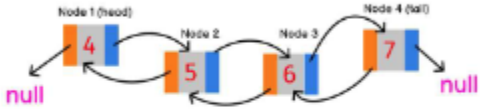
In your driver function, show the use of each list operation and show the output in table 3-3 found in section 6 with a descriptive caption for each. The tasks you have to perform are as follows:

a. Traverse the list by passing the head of the created list into the function
b. Insert the element 'G' at the start of the list to replace the current node. Output should now show "GCPE101"
c. Insert an element "E" with the previous node element being "P". Output should now show "GCPEE101".
d. Delete the node containing the element C.

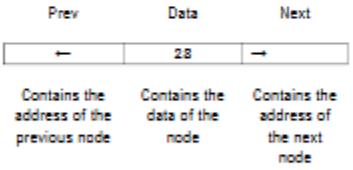
e. Delete the node containing the element P.
f. Show the elements in the list. Output should be "GEE101".

A.2. Doubly Linked List

The singly linked list allows for direct access from a list node only to the next node in the list. A doubly linked list allows convenient access from a list node to the next node and also to the preceding node on the list.



The doubly linked list node accomplishes this in the obvious way by storing two pointers: one to the node following it (as in the singly linked list), and a second pointer to the node preceding it.



This means that in addition to our implementation of the singly linked list, we'll have addition compartment.

```
class Node{
public:
    char data;
    Node *next;
    Node *prev;
};
```

Modify the given operations used in the singly linked lists to work on the new construct of a doubly linked list. Provide a screenshot of your code and analysis in table 3-4.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

HOA_3_Reyes > main.cpp > main()

```cpp
#include <iostream>
#include "doubly_ll.h"

using namespace std;

int main()
{
    // Create the list: CPE010
    DoubleList<char>* head = createNode('C');
    DoubleList<char>* second = createNode('P');
    DoubleList<char>* third = createNode('E');
    DoubleList<char>* fourth = createNode('0');
    DoubleList<char>* fifth = createNode('1');
    DoubleList<char>* last = createNode('0');

    // Connect the nodes
    head->next = second;
    second->prev = head;

    second->next = third;
    third->prev = second;

    third->next = fourth;
    fourth->prev = third;

    fourth->next = fifth;
    fifth->prev = fourth;

    fifth->next = last;
    last->prev = fifth;

    cout << "Original List: ";
    dllTraverse(head);

    // Insert G at the head
    cout << "\nInsert G at Head" << endl;
    dllInsertHead('G', head);
    dllTraverse(head);

    // Insert E after P
    cout << "\nInsert E after P" << endl;

    DoubleList<char>* current = head;
    while (current != nullptr)
    {
        if (current->data == 'P')
        {
            dllInsertAfter(current, 'E');
```

```
            break;
        }
        current = current->next;
    }

    dllTraverse(head);

    // Insert X at the end
    cout << "\nInsert X at End" << endl;
    dllInsertEnd('X', head);
    dllTraverse(head);

    // Delete C
    cout << "\nDelete C" << endl;
    dllDelete('C', head);
    dllTraverse(head);

    // Delete P
    cout << "\nDelete P" << endl;
    dllDelete('P', head);
    dllTraverse(head);

    return 0;
}
```

doubly_ll.h | singly_ll.h

HOA_3_Reyes > doubly_ll.h > ...

```cpp
#ifndef DOUBLY_LL_H
#define DOUBLY_LL_H

#include <iostream>
using namespace std;

// Doubly Linked List Node
template <typename T>
class DoubleList
{
public:
    T data;
    DoubleList<T>* next;
    DoubleList<T>* prev;
};

// Create a new node
template <typename T>
DoubleList<T>* createNode(T newData)
{
    DoubleList<T>* newNode = new DoubleList<T>;

    newNode->data = newData;
    newNode->next = nullptr;
    newNode->prev = nullptr;

    return newNode;
}

// Traverse the list
template <typename T>
void dllTraverse(DoubleList<T>* head)
{
    if (head == nullptr)
    {
        cout << "The list is empty." << endl;
        return;
    }

    DoubleList<T>* current = head;

    cout << "List: ";

    while (current != nullptr)
    {
        cout << current->data << " ";
        current = current->next;
    }
```

```
49
50      cout << endl;
51  }
52
53  // Insert at Head
54  template <typename T>
55  void dllInsertHead(T value, DoubleList<T>*& head)
56  {
57      DoubleList<T>* newNode = createNode(value);
58
59      newNode->next = head;
60
61      if (head != nullptr)
62          head->prev = newNode;
63
64      head = newNode;
65  }
66
67  // Insert at End
68  template <typename T>
69  void dllInsertEnd(T value, DoubleList<T>*& head)
70  {
71      DoubleList<T>* newNode = createNode(value);
72
73      if (head == nullptr)
74      {
75          head = newNode;
76          return;
77      }
78
79      DoubleList<T>* current = head;
80
81      while (current->next != nullptr)
82          current = current->next;
83
84      current->next = newNode;
85      newNode->prev = current;
86  }
87
88  // Insert After
89  template <typename T>
90  void dllInsertAfter(DoubleList<T>* prevNode, T value)
```

```
{
    if (prevNode == nullptr)
        return;

    DoubleList<T>* newNode = createNode(value);

    newNode->next = prevNode->next;
    newNode->prev = prevNode;

    if (prevNode->next != nullptr)
        prevNode->next->prev = newNode;

    prevNode->next = newNode;
}

// Delete a Node
template <typename T>
void dllDelete(T value, DoubleList<T>*& head)
{
    if (head == nullptr)
        return;

    DoubleList<T>* current = head;

    while (current != nullptr && current->data != value)
        current = current->next;

    if (current == nullptr)
        return;

    if (current == head)
        head = current->next;

    if (current->next != nullptr)
        current->next->prev = current->prev;

    if (current->prev != nullptr)
        current->prev->next = current->next;

    delete current;
}

#endif
```

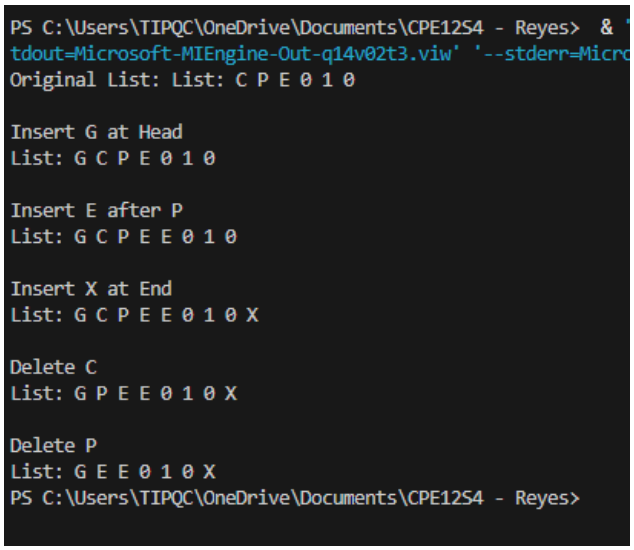
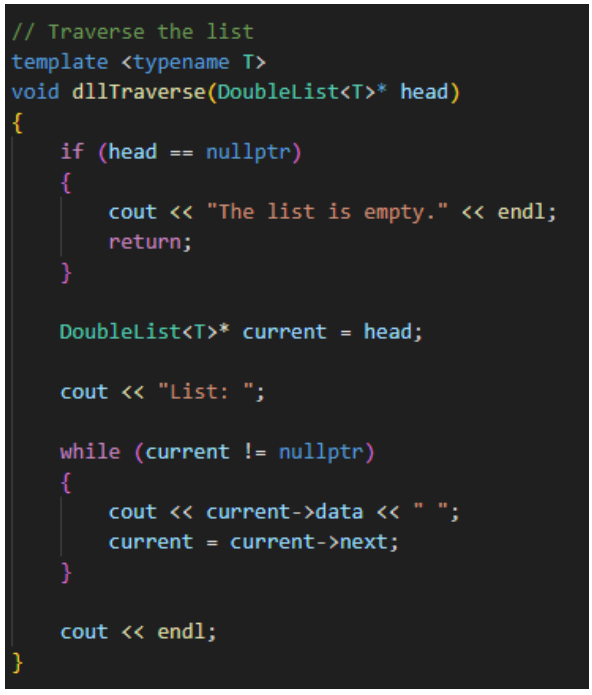
Screenshot	
Discussion	The program successfully created a doubly linked list representing the string **CPE010**. Each node was dynamically allocated using the **createNode()** function and connected using both the **next** and **prev** pointers. The **next** pointer links each node to the following node, while the **prev** pointer links it to the previous node. The traversal function displayed the contents of the list in the correct order, confirming that all nodes were properly connected.

Table 3-1. Output of Initial/Simple Implementation

Operation	**Screenshot**
Traversal	

```
// Traverse the list
template <typename T>
void dllTraverse(DoubleList<T>* head)
{
    if (head == nullptr)
    {
        cout << "The list is empty." << endl;
        return;
    }

    DoubleList<T>* current = head;

    cout << "List: ";

    while (current != nullptr)
    {
        cout << current->data << " ";
        current = current->next;
    }

    cout << endl;
}
```

Insertion at head

```cpp
// Insert at Head
template <typename T>
void dllInsertHead(T value, DoubleList<T>*& head)
{
    DoubleList<T>* newNode = createNode(value);

    newNode->next = head;

    if (head != nullptr)
        head->prev = newNode;

    head = newNode;
}
```

Insertion at any part of the list

```cpp
// Insert After
template <typename T>
void dllInsertAfter(DoubleList<T>* prevNode, T value)
{
    if (prevNode == nullptr)
        return;

    DoubleList<T>* newNode = createNode(value);

    newNode->next = prevNode->next;
    newNode->prev = prevNode;

    if (prevNode->next != nullptr)
        prevNode->next->prev = newNode;

    prevNode->next = newNode;
}
```

Insertion at the end

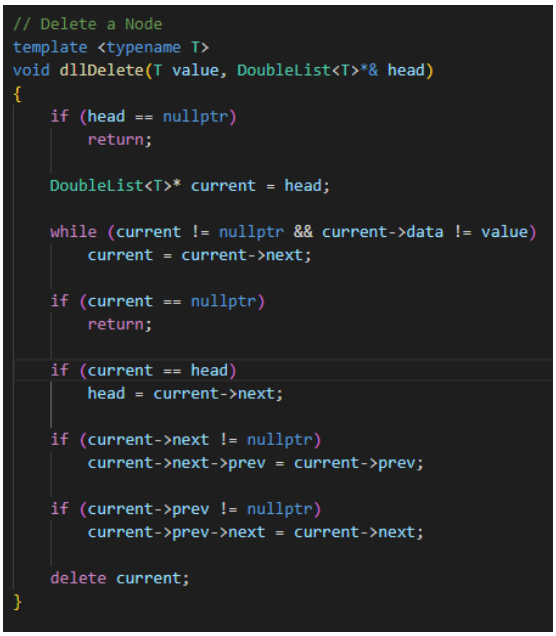
```cpp
// Insert at End
template <typename T>
void dllInsertEnd(T value, DoubleList<T>*& head)
{
    DoubleList<T>* newNode = createNode(value);

    if (head == nullptr)
    {
        head = newNode;
        return;
    }

    DoubleList<T>* current = head;

    while (current->next != nullptr)
        current = current->next;

    current->next = newNode;
    newNode->prev = current;
}
```

Deletion of a node	

```
// Delete a Node
template <typename T>
void dllDelete(T value, DoubleList<T>*& head)
{
    if (head == nullptr)
        return;

    DoubleList<T>* current = head;

    while (current != nullptr && current->data != value)
        current = current->next;

    if (current == nullptr)
        return;

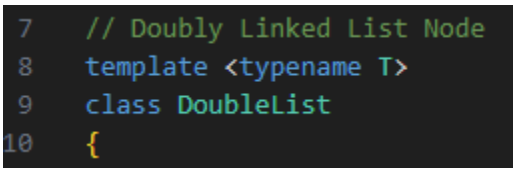
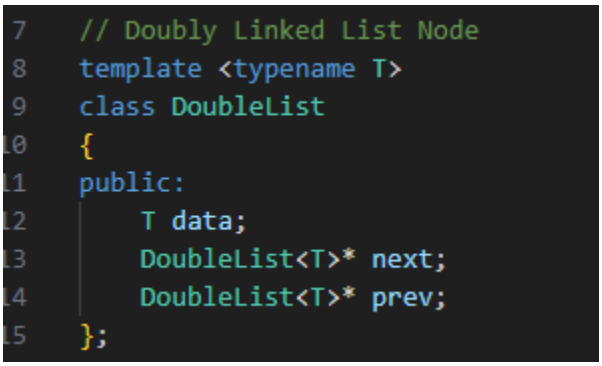
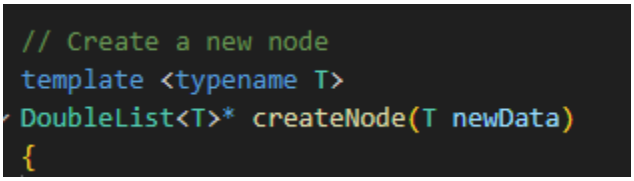
    if (current == head)
        head = current->next;

    if (current->next != nullptr)
        current->next->prev = current->prev;

    if (current->prev != nullptr)
        current->prev->next = current->next;

    delete current;
}
```

Table 3-2. Code for the List Operations

Source code	
consol	
	

```
// Doubly Linked List Node
template <typename T>
class DoubleList
{
```

```
// Doubly Linked List Node
template <typename T>
class DoubleList
{
public:
    T data;
    DoubleList<T>* next;
    DoubleList<T>* prev;
};
```

```
// Create a new node
template <typename T>
DoubleList<T>* createNode(T newData)
{
```

```cpp
// Create a new node
template <typename T>
DoubleList<T>* createNode(T newData)
{
    DoubleList<T>* newNode = new DoubleList<T>;

    newNode->data = newData;
    newNode->next = nullptr;
    newNode->prev = nullptr;

    return newNode;
}
```

```cpp
// Traverse the list
template <typename T>
void dllTraverse(DoubleList<T>* head)
{
```

```cpp
// Traverse the list
template <typename T>
void dllTraverse(DoubleList<T>* head)
{
    if (head == nullptr)
    {
        cout << "The list is empty." << endl;
        return;
    }

    DoubleList<T>* current = head;

    cout << "List: ";

    while (current != nullptr)
    {
        cout << current->data << " ";
        current = current->next;
    }

    cout << endl;
}
```

```cpp
// Insert at Head
template <typename T>
void dllInsertHead(T value, DoubleList<T>*& head)
{
```

```cpp
// Insert at Head
template <typename T>
void dllInsertHead(T value, DoubleList<T>*& head)
{
    DoubleList<T>* newNode = createNode(value);

    newNode->next = head;

    if (head != nullptr)
        head->prev = newNode;

    head = newNode;
}
```

```cpp
// Insert at End
template <typename T>
void dllInsertEnd(T value, DoubleList<T>*& head)
{
```

```cpp
// Insert at End
template <typename T>
void dllInsertEnd(T value, DoubleList<T>*& head)
{
    DoubleList<T>* newNode = createNode(value);

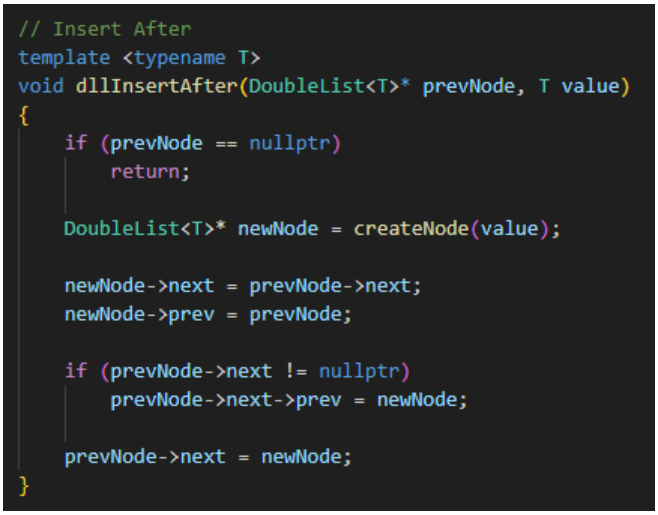
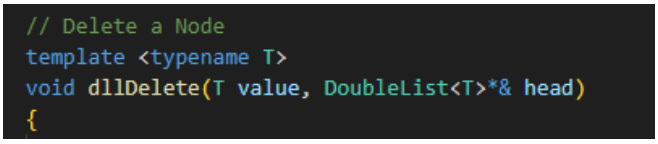
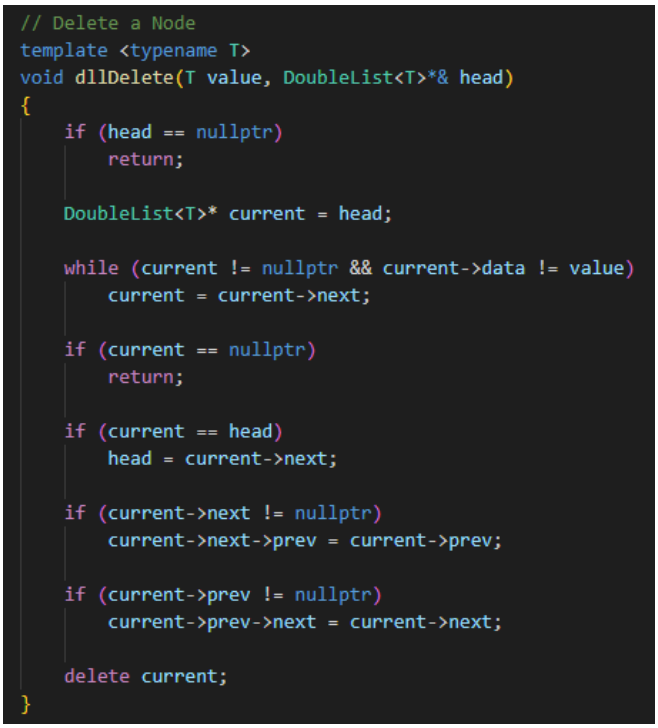
    if (head == nullptr)
    {
        head = newNode;
        return;
    }

    DoubleList<T>* current = head;

    while (current->next != nullptr)
        current = current->next;

    current->next = newNode;
    newNode->prev = current;
}
```

```cpp
// Insert After
template <typename T>
void dllInsertAfter(DoubleList<T>* prevNode, T value)
{
```

```
// Insert After
template <typename T>
void dllInsertAfter(DoubleList<T>* prevNode, T value)
{
    if (prevNode == nullptr)
        return;

    DoubleList<T>* newNode = createNode(value);

    newNode->next = prevNode->next;
    newNode->prev = prevNode;

    if (prevNode->next != nullptr)
        prevNode->next->prev = newNode;

    prevNode->next = newNode;
}
```

```
// Delete a Node
template <typename T>
void dllDelete(T value, DoubleList<T>*& head)
{
```

```
// Delete a Node
template <typename T>
void dllDelete(T value, DoubleList<T>*& head)
{
    if (head == nullptr)
        return;

    DoubleList<T>* current = head;

    while (current != nullptr && current->data != value)
        current = current->next;

    if (current == nullptr)
        return;

    if (current == head)
        head = current->next;

    if (current->next != nullptr)
        current->next->prev = current->prev;

    if (current->prev != nullptr)
        current->prev->next = current->next;

    delete current;
}
```

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshot(s)	Analysis

Table 3-4. Modified Operations for Doubly Linked Lists

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.